

Занятие 2. Как редактировать конечную память: decay, delta rule, gates

Как записывать новое в state фиксированного размера, не превращая его в бесконечную сумму помех?

Автор: Андрей Ильинский, 2026

Из занятия 1 нужны две формулы

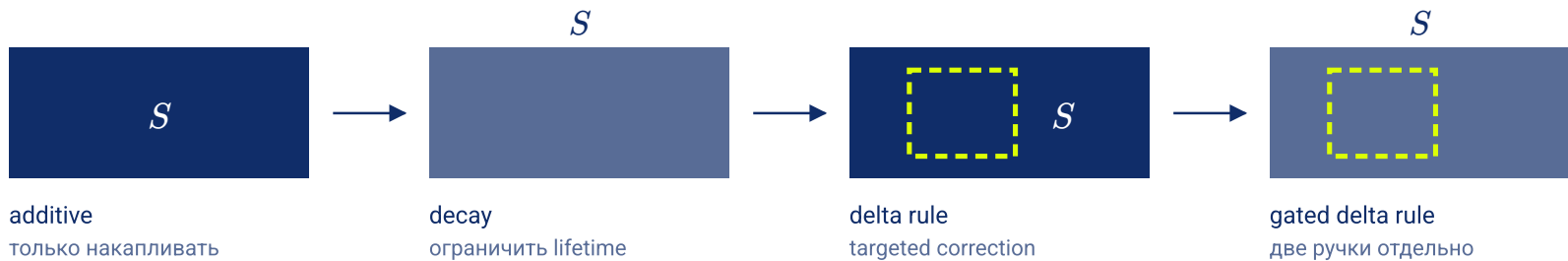
$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$



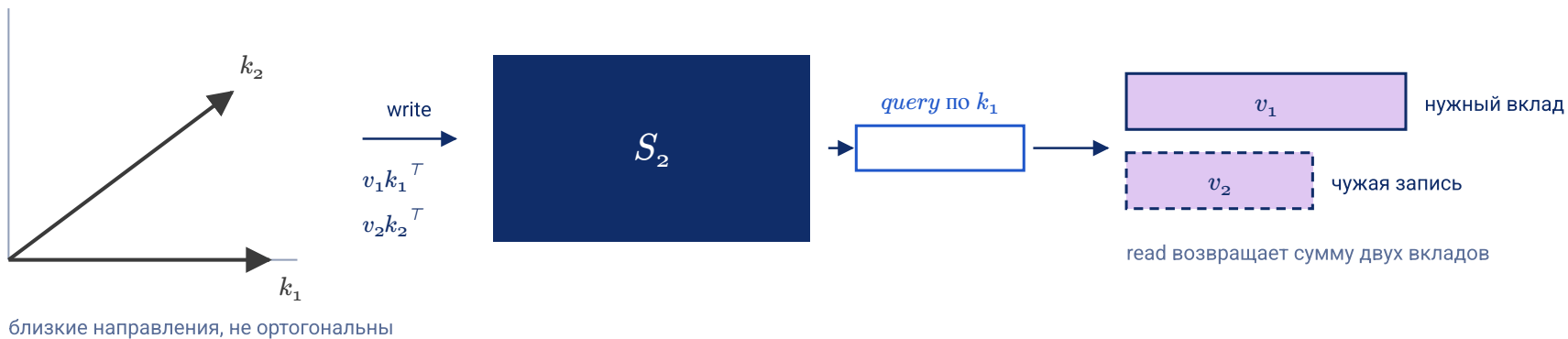
Что если записанные keys неортогональны?

Линия управления памятью

additive state $\rightarrow$ decay $\rightarrow$
delta rule $\rightarrow$ gated delta rule.



Interference: похожий key читает чужую запись



Откуда берётся interference

$$S_{t-1} = \sum_{i=1}^{t-1} v_i k_i^\top.$$

$$\hat{v}_t^\top = \sum_{i=1}^{t-1} (k_t^\top k_i) v_i^\top.$$

$$\hat{v}_t^\top = (k_t^\top k_j) v_j^\top + \sum_{\substack{i=1 \\ i \neq j}}^{t-1} (k_t^\top k_i) v_i^\top.$$

Подсвеченная сумма и есть interference: она обнуляется, когда остальные keys ортогональны текущему.

Одна чужая запись уже меняет read

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$k_2 = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

$$S_2 = \begin{bmatrix} 1 & 0 \\ 0.8 & 0.6 \end{bmatrix}.$$

$$S_2 k_1 = \begin{bmatrix} 1 \\ 0.8 \end{bmatrix}.$$

правильный вклад
[1, 0]

лишний вклад
[0, 0.8]

Fixed shape $\neq$ unlimited memory

fixed number of state scalars, growing number of write attempts.



Три разных утверждения

модель **принимает** 1M токенов — свойство буфера



модель **не ломается** на 1M — выдаёт связный текст



модель **использует** начало — извлекает то, что там лежало

Каждое следующее утверждение не следует из предыдущего.

KV-кэш хранит K и V прошлых позиций отдельно. Состояние фиксированного размера сжимает всю историю.

MQAR против needle

А 4 В 3 С 6 F 1 ...

А ? → 4

С ? → 6

ЧТО МЕРЯЕМ	ЧЕМ	ЧТО УСТАНОВЛИВАЕТ
ёмкость памяти	многозапросный recall	как качество падает с числом пар
дальнее извлечение одного факта	needle in a haystack	достаёт вставленный факт с данной позиции
прикладное качество	наборы задач	эффект на своём распределении

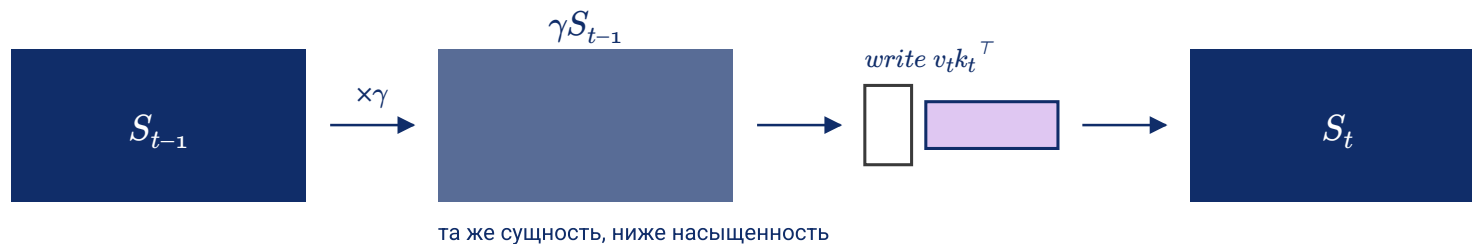
Один запрос — не одна хранимая пара: MQAR добавляет много recall-событий на разных расстояниях. Ключевое слово — multi.

Первое управление: забывать старое

Можно ли ограничить lifetime записей, не меняя shape state?

Fixed decay

$$S_t = \gamma S_{t-1} + v_t k_t^\top, \quad 0 \leq \gamma \leq 1.$$



Что хранит decayed state

$$S_3 = \gamma^2 v_1 k_1^\top + \gamma v_2 k_2^\top + v_3 k_3^\top.$$

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$



$$\gamma^2 v_1 k_1^\top$$



$$\gamma v_2 k_2^\top$$



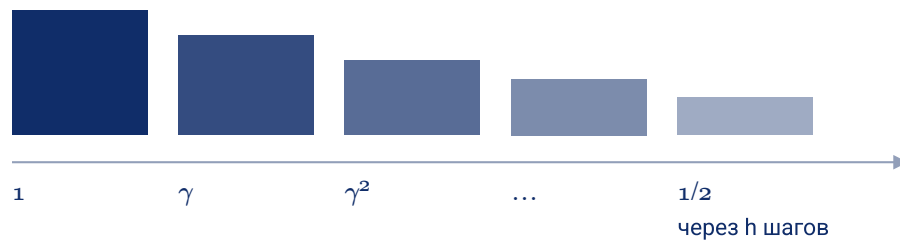
$$v_3 k_3^\top$$

старое слабее, новое в полную силу

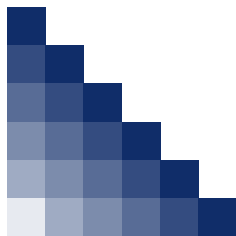
Half-life памяти

$$\gamma^h = \frac{1}{2}.$$

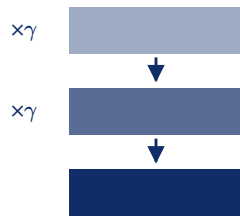
$$h = \frac{\log(1/2)}{\log \gamma}.$$



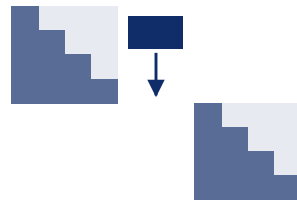
RetNet: тот же decayed operator в трёх формах



parallel



recurrent



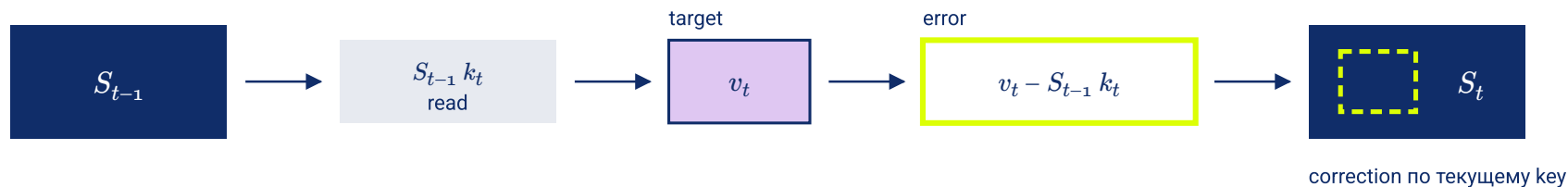
chunkwise

$$R_{ti}^{(\gamma)} = \mathbf{1}[i \leq t] \gamma^{t-i}, \quad S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

Decay меняет lifetime, но не исправляет конкретную association

Можно ли сначала прочесть старое значение, а записать только ошибку?

Delta rule = read → error → correction



$$S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top.$$

Delta rule как gradient descent: локальный loss

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|_2^2.$$

prediction

Sk_t — что память отдаёт сейчас

target

v_t — что должно быть по этому key

Gradient по matrix memory

$$e_b = \sum_{a=1}^{d_k} S_{ba}(k_t)_a - (v_t)_b.$$

$$\frac{\partial \ell_t}{\partial S_{ba}} = e_b(k_t)_a.$$

Вместо скалярного loss появилось направление: $\nabla_S \ell_t = e k_t^\top$ – внешнее произведение ошибки и key.

Один gradient step даёт delta update

$$S_t = S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^\top.$$

$$S_t = S_{t-1} + \beta_t (v_t - S_{t-1} k_t) k_t^\top.$$

Optimizer действует на fast state, не на network weights.

Delta rule

Как записать в память исправление, а не ещё одну ассоциацию?

Delta rule = erase + write

$$S_t = S_{t-1} + \beta_t v_t k_t^\top - \beta_t S_{t-1} k_t k_t^\top.$$

write

$$+\beta_t v_t k_t^\top$$

erase

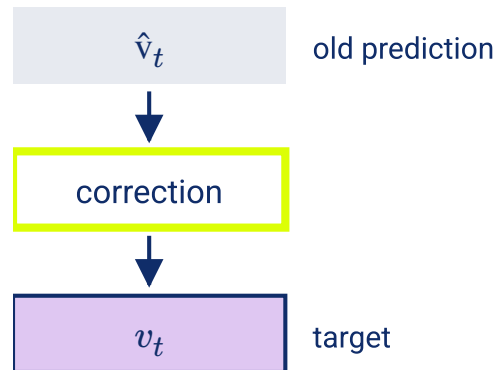
$$-\beta_t S_{t-1} k_t k_t^\top$$

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Когда correction становится exact по текущему key

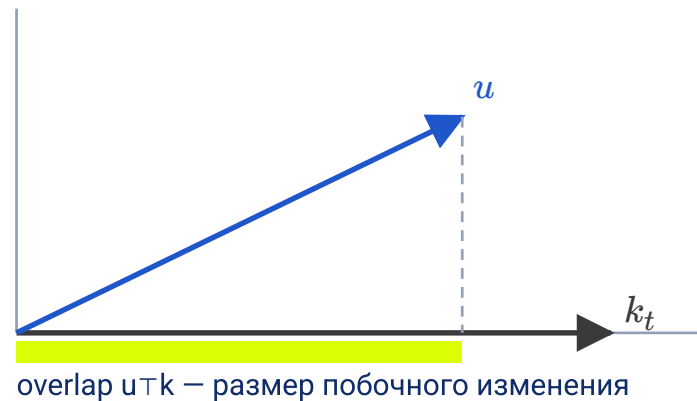
$$S_t k_t = \hat{v}_t + \beta_t \|k_t\|_2^2 (v_t - \hat{v}_t).$$

$$\beta_t = \frac{1}{\|k_t\|_2^2}.$$



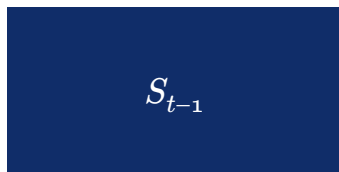
Targeted не означает isolated

$$S_t u = S_{t-1} u + \beta_t (v_t - \hat{v}_t) (k_t^\top u).$$



Additive write против delta correction

additive



old value остаётся
new value кладётся сверху

delta rule



erase по направлению key



write только correction

Нужны две независимые ручки

Как отдельно управлять глобальным forgetting и targeted correction?

Gated DeltaNet

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

α_t — global retention

насыщенность состояния

β_t — targeted correction

контур записи, не заливка



От двух операций к одной формуле

$$S_t = \alpha_t S_{t-1} + \beta_t (v_t - \alpha_t S_{t-1} k_t) k_t^\top.$$

$$S_t = \alpha_t S_{t-1} + \beta_t v_t k_t^\top - \alpha_t \beta_t S_{t-1} k_t k_t^\top.$$

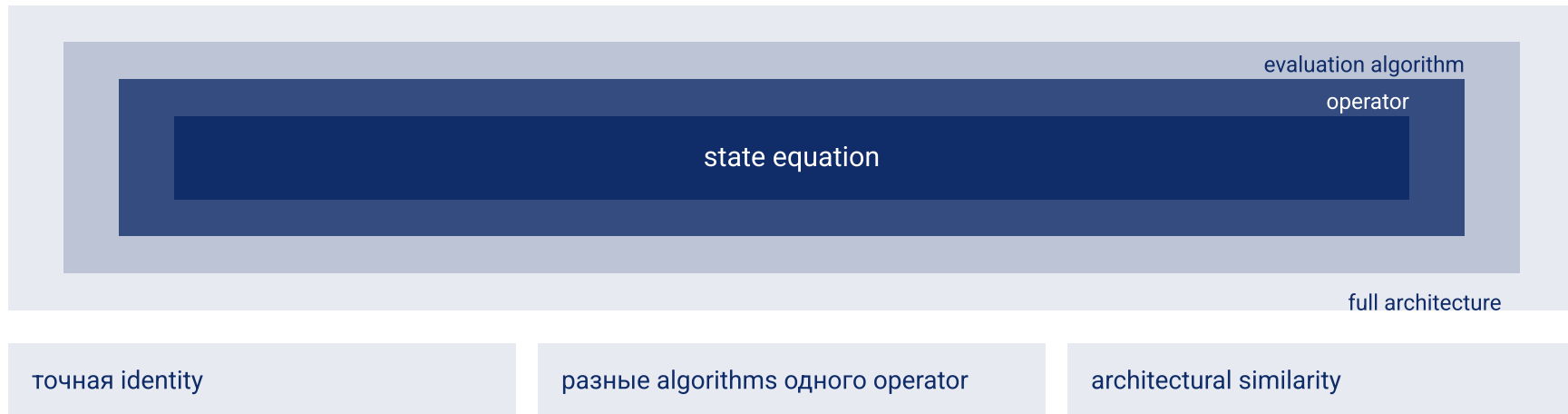
$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Одно и то же правило: сначала как две операции, затем как один transition operator.

Четыре режима управления state

MECHANISM	OLD STATE	WRITE	ЧТО УМЕЕТ
Additive	оставить полностью	$v_t k_t^\top$	только накапливать
Fixed decay	умножить на γ	$v_t k_t^\top$	ограничивать lifetime
Delta rule	key-directed erase	prediction error	targeted correction
Gated DeltaNet	global α_t + key-directed erase	β_t correction	разделить forgetting и editing

Похожая recurrence ещё не означает одну architecture



**Delta rule исправляет чтение по текущему ключу.
Другие неортогональные направления при этом
меняются.**

Finite memory стала редактируемой



additive



decay



delta edit



gated delta

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Какие реальные architectures выбирают разные transition, gate и evaluation algorithm?